

VOLUME 7

COMPLETE VOLUME — CHAPTERS 7.1–7.13

Development Environment

A Reproducible, Windows-First Setup From Bare Machine to Running App

Document Title	Volume 7 — Development Environment (Combined)
Volume Reference	Volume 7
Chapters Included	7.1 through 7.13 (13 chapters)
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026
Governed By	Volume 0 — Project Foundation; grounded in Volumes 3, 4, and 6

Table of Contents

This volume takes a bare Windows machine to a fully working, reproducible development environment for the Human Archive app — hardware requirements, every piece of software, editor/IDE setup, Flutter's pinned-version installation, Git and its hooks, Firebase and AWS provisioning, FFmpeg as a dev-only tool, environment variables, project initialization, dependency installation, and a final end-to-end verification checklist.

Chapter	Title
7.1	Development Machine Requirements
7.2	Software Installation
7.3	VS Code Setup
7.4	Android Studio Setup
7.5	Flutter Installation
7.6	Git Setup
7.7	Firebase Setup
7.8	AWS Setup
7.9	FFmpeg Setup
7.10	Environment Variables
7.11	Project Initialization
7.12	Dependency Installation
7.13	Verification Checklist

VOLUME 7 — DEVELOPMENT ENVIRONMENT

CHAPTER 7.1

Development Machine Requirements

What the Windows Machine Actually Needs to Support This Stack

Document Title	Development Machine Requirements
Chapter Reference	7.1
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026
Governed By	Volume 0, Chapter 0.2 — Windows-first, 90–100% of development here.

1. Minimum Hardware

Resource	Minimum	Why
OS	Windows 10/11, 64-bit	Required by Flutter's Windows tooling and Android Studio.
RAM	16 GB (32 GB recommended)	Android emulator + Android Studio + VS Code + Chrome-based tooling running concurrently is genuinely memory-hungry; 8GB will work but with visible friction.
Disk	60 GB free	Android SDK + emulator images + Flutter SDK + build caches (Chapter 3.8's pinned versions) add up quickly.
CPU	Virtualization-capable (Intel VT-x / AMD-V)	Required for the Android emulator (Chapter 7.4) to run at usable speed — without it, only a physical Android device works, which is a valid fallback but not the default.

2. Required Enablement Before Software Installation

- Hyper-V or Windows Hypervisor Platform enabled (Windows Features) — needed for Android emulator acceleration.
- Developer Mode enabled in Windows Settings — required for Flutter's symlink support during build.
- Administrator rights for the initial install session (Chapter 7.2) — not required for day-to-day development afterward.

3. Physical Android Device (Recommended, Not Required)

Volume 0 already names Android as the primary test target. A physical device (USB debugging enabled) gives more reliable camera/wide-angle-lens testing (Volume 5, Chapter 5.1's capability ladder) than the emulator, which cannot fully simulate real camera hardware — recommended for anyone actively testing the Recording Engine (Volume 5), optional for anyone working only on Admin-side screens.

4. iOS — Explicitly Not Local

No Mac required on this machine: per Volume 0's Constitution, iOS build/sign/test happens entirely via the cloud Mac service configured in Chapter 7.7/7.8 credentials — this Windows machine never needs Xcode installed.

CHAPTER 7.2

Software Installation

Everything Installed Before Any Project-Specific Setup Begins

Document Title	Software Installation
Chapter Reference	7.2
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026

1. Install Order

Installed in this order, since later items (Flutter, Chapter 7.5) check for earlier ones (Git, JDK) during their own setup/doctor checks:

#	Software	Purpose
1	Git for Windows	Version control (Chapter 7.6).
2	A JDK (Temurin 17 LTS)	Required by the Android Gradle build, independent of Android Studio's own bundled JDK.
3	Android Studio	Android SDK, platform tools, emulator (Chapter 7.4).
4	Visual Studio Code	Primary editor (Chapter 7.3).
5	Flutter SDK via fvm	Chapter 7.5 — installed after Android Studio so its SDK path is already available for flutter doctor to detect.
6	FFmpeg	Dev-tooling only, not shipped in the app (Chapter 7.9).
7	AWS CLI	Local testing against the Chapter 4.9 cloud topology (Chapter 7.8).

2. Why fvm, Not a Bare Flutter Install

Volume 3, Chapter 3.8 already fixed the policy: the Flutter SDK is pinned via fvm, not installed as a single loose "stable channel" copy — this is set up in full in Chapter 7.5, but the tool itself (fvm, via a pub global activation) is installed at this stage alongside everything else.

3. Verification

- `git --version`, `java -version`, and flutter doctor (once Chapter 7.5 completes) are the three commands used throughout this volume to confirm each piece landed correctly — Chapter 7.13's checklist is where all of them are run together as a final gate.

VOLUME 7 — DEVELOPMENT ENVIRONMENT

CHAPTER 7.3

VS Code Setup

Extensions and Settings for the Primary Editor

Document Title	VS Code Setup
Chapter Reference	7.3
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive

Prepared With	Claude (Cowork)
Date	05 August 2026

1. Required Extensions

Extension	Purpose
Flutter (Dart-Code.flutter)	Debugging, hot reload, widget inspector — installs the Dart extension as a dependency automatically.
Error Lens	Surfaces the Chapter 3.7 lint rules inline, not just in the Problems panel — makes the 'static analysis must run clean' rule hard to ignore accidentally.
GitLens	Blame/history context, useful for the traceability comments Chapter 3.7 requires (linking a line back to a requirement ID).
EditorConfig for VS Code	Enforces consistent indentation/line-ending across contributors, backing the .editorconfig committed in Chapter 6.2's project structure.

2. Project-Level settings.json

```
{
  "editor.formatOnSave": true,
  "[dart]": {
    "editor.defaultFormatter": "Dart-Code.dart-code",
    "editor.rulers": [80]
  },
  "dart.lineLength": 80,
  "dart.previewFlutterUiGuides": true
}
```

3. Launch Configurations

```
// .vscode/launch.json
{
  "configurations": [
    {
      "name": "human_archive_app (Android, dev)",
      "request": "launch", "type": "dart",
      "args": ["--flavor", "dev", "--dart-define-from-file=.env.dev"]
    }
  ]
}
```

The `--dart-define-from-file` flag is how Chapter 7.10's environment variables actually reach a debug build — set up here so a fresh contributor can press F5 and get a correctly-configured dev build immediately, without a manual step.

4. What This Chapter Defers

- The `.env.dev` file's actual contents — Chapter 7.10.
- Android Studio's parallel setup, for contributors who prefer it — Chapter 7.4.

Android Studio Setup

SDK Components and the Emulator Profile This Project Actually Needs

Document Title	Android Studio Setup
Chapter Reference	7.4
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026

1. SDK Manager Components

- Android SDK Platform matching the app's compileSdkVersion (set once Volume 6's project is initialized, Chapter 7.11) and at least the two most recent stable API levels for testing across OS versions.
- Android SDK Build-Tools (latest stable).
- Android Emulator + at least one system image with Google Play services (needed since Firebase Auth/FCM, Volume 4 Chapter 4.1, requires Play Services on the emulator).
- Android SDK Command-line Tools — required for Flutter's own tooling to invoke sdkmanager/avdmanager under the hood.

2. Recommended AVD (Emulator) Profile

Setting	Value	Why
Device	Pixel 6 or similar mid-range profile	Representative of the actual Collector fleet better than a top-of-line device would be.
System Image	Latest stable, Google Play (not Google APIs-only)	Google Play Services required for Firebase (Volume 4.1).
RAM	4 GB	Matches Chapter 7.1's overall machine budget without starving the host.
Camera (front/back)	Emulated / Webcam	The emulator's simulated camera cannot test Volume 5.1's real wide-angle lens selection — this is why Chapter 7.1 recommends a physical device for Recording Engine work.

3. Known Limitation Carried From Volume 5

Camera capability ladder (Volume 5, Chapter 5.1) cannot be fully verified on the emulator: the emulator has no real ultra-wide sensor, so Tier 1/2 of the fallback ladder can only be exercised on real hardware. Emulator testing is sufficient for every other feature module (Volume 3, Chapter 3.5) — Projects/Tasks, Admin screens, upload queue logic against a mocked repository — but Recording Engine acceptance testing requires the physical device from Chapter 7.1.

4. What This Chapter Defers

- Installing Flutter itself and running flutter doctor against this SDK — Chapter 7.5.

Flutter Installation

Installing the Exact Pinned SDK Version via fvm

Document Title	Flutter Installation
Chapter Reference	7.5
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026
Implements	Volume 3, Chapter 3.8's version-pinning policy

1. Install fvm, Then the Pinned Version

```
dart pub global activate fvm
fvm install <pinned-version>      # the exact version in .fvmrc, Ch.6.2
fvm use <pinned-version>          # sets this project's .fvmrc
fvm flutter doctor
```

Every contributor runs `fvm flutter` (not a bare `flutter` command) so the SDK version is always the one pinned in `.fvmrc` (Chapter 6.2) — this is what Chapter 3.8 meant by 'all builds against the exact same Flutter version.'

2. PATH Setup (Windows)

Do not add a global Flutter SDK to PATH: `fvm` intentionally avoids that — instead, add `fvm`'s own shim directory to `PATH`, and IDEs (Chapter 7.3/7.4) are pointed at the `fvm`-managed SDK path per-project, so two projects on the same machine could even use different Flutter versions without conflict (not a current need, but a free side benefit of doing this correctly from the start).

3. flutter doctor — Expected Clean Output

- [✓] Flutter (fvm-managed, pinned version)
- [✓] Android toolchain (from Chapter 7.4)
- [✓] Chrome (used for one-off web-rendering debug tools; the app itself is not a web target)
- [✓] Android Studio
- [✓] VS Code (from Chapter 7.3)
- [!] Xcode / iOS toolchain — expected and acceptable to show as unavailable, since iOS build/sign/test happens on the cloud Mac service (Chapter 7.1, Section 4), never on this machine.

4. What This Chapter Defers

- Cloning the actual project and running its own setup — Chapter 7.11.
- Package/dependency installation — Chapter 7.12.

VOLUME 7 — DEVELOPMENT ENVIRONMENT

CHAPTER 7.6

Git Setup

Repository, Branching, and the Hooks That Enforce Chapter 3.7

Document Title	Git Setup
Chapter Reference	7.6
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026

1. Branch Strategy

- main is always deployable; feature branches (feature/<short-description>) are short-lived and merged via pull request, never pushed to directly.
- Every PR is checked against Volume 3, Chapter 3.7, Section 9's review checklist before merge — this chapter's git hooks (Section 2) automate the mechanical parts of that checklist so reviewers focus on the judgment calls, not formatting.

2. Pre-Commit Hook

```
#!/bin/sh
fvm flutter format --set-exit-if-changed .
fvm flutter analyze
```

Installed via a tool like lefthook or husky-equivalent for Dart, so 'static analysis must run clean' (Volume 0, Chapter 0.2) is enforced before a commit even lands locally, not only in CI — catching mistakes at the cheapest possible point.

3. .gitignore Highlights

- build/, .dart_tool/, and IDE-specific folders (.idea/, minus shared launch configs already committed per Chapter 7.3) are ignored.
- Generated *.g.dart files are explicitly NOT ignored (Volume 3, Chapter 3.6, Section 5) — this is worth restating here since it's the opposite of many Flutter starter templates and easy to get wrong by copying a generic .gitignore.
- .env* files (real ones, not .env.example) are ignored — see Chapter 7.10; secrets never enter version control.

4. Commit Messages

Conventional, imperative-mood messages referencing the requirement/chapter they implement where relevant (e.g. "Add checklist retry action (FR-CHK-05, C-08)") — extending Chapter 3.7's traceability-comment rule from code into the commit history itself.

VOLUME 7 — DEVELOPMENT ENVIRONMENT

CHAPTER 7.7

Firebase Setup

Provisioning Auth, FCM, and Crashlytics for Every Environment

Document Title	Firestore Setup
Chapter Reference	7.7
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026
Provisions	Volume 4, Chapter 4.1 (Auth/FCM); Volume 6, Chapter 6.9 (Crashlytics, ADR-010)

1. Three Firebase Projects, Not One

Matching Volume 4, Chapter 4.9's dev/staging/prod separation exactly — a bug in a dev build must never be able to send a real push notification to a production Collector's device or pollute production Crashlytics data.

Environment	Firebase Project	Used By
dev	human-archive-dev	Local development builds (Chapter 7.3's launch config).
staging	human-archive-staging	Pre-release QA builds.
prod	human-archive-prod	Released builds only.

2. Steps Per Project

- 1. Create the Firebase project in the console.
- 2. Enable Authentication → Email/Password provider (matching SH-02's MVP scope, Chapter 2.7).
- 3. Enable Cloud Messaging (Volume 6, Chapter 6.8).
- 4. Enable Crashlytics (Volume 6, Chapter 6.9, ADR-010).
- 5. Register the Android app (package name matches the flavor, Chapter 7.11) and download google-services.json into android/app/src/{flavor}/.
- 6. Register the iOS app and download GoogleService-Info.plist — added to the iOS project via the cloud Mac build service (Chapter 7.8's credential handoff), since this machine never opens Xcode directly.

3. Service Account for Backend Verification

A Firebase Admin SDK service account key is generated per environment for the backend's token-verification step (Volume 4, Chapter 4.7) — stored as an encrypted secret in the CI/CD system (Chapter 7.13's pipeline), never committed to the repository, matching Chapter 7.6's .gitignore rule for anything secret.

4. What This Chapter Defers

- Where the resulting config values are referenced from app code — Chapter 7.10 (Environment Variables).

CHAPTER 7.8

AWS Setup

Local Access to the Cloud Topology Fixed in Volume 4

Document Title	AWS Setup
Chapter Reference	7.8
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026
Provisions	Volume 4, Chapter 4.9's AWS account topology

1. Per-Developer IAM User

Each developer gets their own IAM user (never a shared root or shared access key), scoped to the dev environment only, with permissions limited to what local development actually needs: invoking the dev API Gateway/Lambda stack, reading/writing the dev S3 bucket, and read-only access to dev Aurora logs — matching Chapter 4.9's least-privilege principle at the human level, not just the service level.

2. AWS CLI Configuration

Profile	Purpose
human-archive-dev	The only profile most contributors need day-to-day.
human-archive-staging / -prod	Restricted to release engineers only, configured separately, not part of a standard onboarding.

3. Local Testing Against AWS

- Lambda functions (Volume 4, Chapter 4.1) are tested locally against the real dev Aurora/S3 resources rather than a full local AWS emulation — simpler to keep in sync with actual cloud behavior for a small team, at the cost of needing network access to iterate (an accepted tradeoff, revisited only if it becomes a real friction point).
- S3 uploads during mobile development (Volume 5, Chapter 5.10) point at the dev bucket via the presigned-URL flow already specified in Volume 4, Chapter 4.10 — no separate local storage stand-in is used, so upload behavior tested locally matches production behavior exactly.

4. Credential Hygiene

Never in the repo: AWS credentials are stored via aws configure (in the OS credential store), never in a project file — consistent with Chapter 7.6's secrets rule.

CI credentials are separate: the CI/CD pipeline (Chapter 7.13) uses its own scoped IAM role, never a developer's personal credentials, so revoking one developer's access never breaks the pipeline.

VOLUME 7 — DEVELOPMENT ENVIRONMENT

CHAPTER 7.9

FFmpeg Setup

A Developer Tool, Not an App Dependency

Document Title	FFmpeg Setup
Chapter Reference	7.9
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026

1. Why FFmpeg Is Here at All

The app itself never uses FFmpeg — Volume 5, Chapter 5.4 already established that recording uses the device's own hardware encoder in real time, and Chapter 5.5 explicitly rejected any re-encoding step. FFmpeg is purely a developer-side diagnostic tool, useful for a few recurring tasks during Recording Engine development and QA.

2. What It's Used For

- Inspecting a captured .mp4 chunk's actual encoded parameters (ffprobe) to verify they match Chapter 5.2's specification (1920×1080, 30fps, H.264, ~8,000kbps) during manual testing.
- Producing small synthetic test video files for widget/integration tests (Volume 9) that need a valid .mp4 fixture without requiring a real device recording.
- Manually verifying a chunk's playability/integrity when debugging a Volume 5, Chapter 5.13 retry/failure report from the field.

3. Installation (Windows)

Recommended: install via a package manager (winget install Gyan.FFmpeg or Chocolatey's ffmpeg package) rather than a manual zip-and-PATH-edit, so it updates cleanly alongside other tooling.

Verification: ffmpeg -version and ffprobe -version both resolve from any terminal once installed — this is the one check Chapter 7.13's final verification checklist runs for this chapter.

4. What This Chapter Explicitly Is Not

- Not a build dependency — pubspec.yaml (Chapter 3.8) never references FFmpeg, and CI (Chapter 7.13) does not need it to build or ship the app, only to run certain test-fixture-generation scripts if a contributor chooses to.

VOLUME 7 — DEVELOPMENT ENVIRONMENT

CHAPTER 7.10

Environment Variables

How Per-Environment Config Reaches the App Without Ever Touching Git

Document Title	Environment Variables
Chapter Reference	7.10
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026

1. Mechanism

Flutter's `--dart-define-from-file`, already wired into Chapter 7.3's VS Code launch config, reading from a per-environment `.env` file that is never committed (Chapter 7.6) — only `.env.example` (committed, with placeholder values documenting what's required) lives in the repository.

2. Required Variables

Variable	Example (dev)	Consumed By
API_BASE_URL	<code>https://dev-api.humanarchive.app/v1</code>	Dio client (Volume 3, ADR-004) — the base for every Volume 4, Chapter 4.6 endpoint call.
FIREBASE_PROJECT_ID	<code>human-archive-dev</code>	Firestore initialization (Chapter 7.7).
S3_BUCKET_NAME	<code>human-archive-dev</code> (Volume 4, Ch.4.10)	Only used for display/debug purposes — the app never talks to S3 directly except via presigned URLs the backend returns, so this is informational, not a credential.
SENTRY_DSN / CRASHLYTICS toggle	n/a — Crashlytics config comes from <code>google-services.json</code>	Listed for completeness; Crashlytics needs no separate dart-define.

3. Per-Flavor Files

```
.env.dev      # matches Ch.7.7's human-archive-dev Firebase project
.env.staging
.env.prod     # release engineers only, per Ch.7.8's access model
```

4. Why Not flutter_dotenv Alone

Compile-time, not runtime: `--dart-define-from-file` bakes values in at build time, which is a stronger guarantee than a runtime-loaded `.env` asset — there is no risk of a production build accidentally reading a dev config file that shipped by mistake, since the values are compiled directly into that specific build.

5. What This Chapter Defers

- How CI supplies these files without committing them — Chapter 7.13, where they're injected as encrypted CI secrets at build time.

VOLUME 7 — DEVELOPMENT ENVIRONMENT

CHAPTER 7.11

Project Initialization

From an Empty Clone to a Scaffolded Volume 3/6 Project

Document Title	Project Initialization
Chapter Reference	7.11
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026
Scaffolds	Volume 3, Chapter 3.6 (Folder Structure); Volume 6, Chapter 6.2 (Project Structure)

1. Steps

```
git clone <repo-url> human_archive_app
cd human_archive_app
fvm use <pinned-version>           # reads .fvsrc, Ch.7.5
cp .env.example .env.dev           # then fill in real dev values, Ch.7.10
```

2. Build Flavors

- dev / staging / prod flavors are configured in android/app/build.gradle and iOS's scheme configuration, each pointing at the matching Firebase project (Chapter 7.7) and API base URL (Chapter 7.10) — a contributor never manually edits config per build; selecting a flavor is enough.

3. First-Run Checklist

- fvm flutter pub get succeeds (Chapter 7.12 covers this in full).
- fvm flutter run --flavor dev launches the app on either the emulator (Chapter 7.4) or a connected physical device (Chapter 7.1) and lands on the Login screen (SH-02).
- Hot reload works after a trivial text change to confirm the toolchain end-to-end.

4. What This Chapter Defers

- Installing/generating the actual package dependencies and generated code — Chapter 7.12.
- The full, final gate confirming every prior chapter's setup actually works together — Chapter 7.13.

VOLUME 7 — DEVELOPMENT ENVIRONMENT

CHAPTER 7.12

Dependency Installation

Pub Packages, Code Generation, and Native Dependencies

Document Title	Dependency Installation
Chapter Reference	7.12
Version	1.0

Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026
Installs	The stack fixed in Volume 3, Chapters 3.1 and 3.8

1. Pub Packages

```
fvm flutter pub get
```

Pulls every package named in Volume 3, Chapter 3.1 (riverpod, drift, dio, go_router, freezed, etc.) at the versions pinned per Chapter 3.8's policy — no manual per-package install step exists outside pubspec.yaml.

2. Code Generation

```
fvm dart run build_runner build --delete-conflicting-outputs
# or, during active development:
fvm dart run build_runner watch --delete-conflicting-outputs
```

Why this step exists at all: riverpod_generator, freezed, and json_serializable (Volume 3, Chapter 3.1) all produce the *.g.dart files that Chapter 6.2 confirmed are committed to the repo — a fresh clone already has working generated code, but this command is still run after pulling any change that touched an @riverpod/freezed-annotated file, and CI (Chapter 7.13) re-runs it to verify the committed generated files are actually up to date.

3. Native/Platform Dependencies

- Android: Gradle sync runs automatically on first fvm flutter run (Chapter 7.11) — no separate manual step on Windows.
- iOS: CocoaPods install (pod install) happens on the cloud Mac build service (Chapter 7.8), never on this Windows machine, consistent with Volume 0's iOS-build-elsewhere rule.

4. Verifying a Clean Install

- fvm flutter analyze returns zero issues (Chapter 3.7's static-analysis rule, checked here as a first practical test rather than just a policy statement).
- fvm flutter test runs the existing test suite (Volume 9) without a single unresolved import error.

VOLUME 7 — DEVELOPMENT ENVIRONMENT

CHAPTER 7.13

Verification Checklist

The Final Gate — Does This Environment Actually Reproduce?

Document Title	Verification Checklist
Chapter Reference	7.13
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026
Purpose	Confirms Volume 7's own stated goal — a reproducible development setup — is actually met, not merely documented.

1. Local Machine Checklist

Check	Command / Action	Expected Result
Hardware/OS	Chapter 7.1's requirements met	Virtualization + Developer Mode enabled.
Core tooling	git --version, java -version	Both resolve (Chapter 7.2).
Flutter	fvm flutter doctor	Clean except the expected iOS/Xcode line (Chapter 7.5).
Project scaffold	fvm flutter pub get, build_runner build	Completes with no errors (Chapter 7.12).
Static analysis	fvm flutter analyze	Zero issues (Chapter 3.7's rule).
Tests	fvm flutter test	Existing suite passes.
App launch — Android	fvm flutter run --flavor dev	Reaches Login (SH-02) on emulator or device.
Firebase connectivity	Login attempt against human-archive-dev	Succeeds or fails with a real auth error, not a config/connection error (Chapter 7.7).
FFmpeg (optional but checked if installed)	ffmpeg -version	Resolves (Chapter 7.9).

2. CI Pipeline Checklist

- A pushed branch triggers analyze + test + build automatically, using the same pinned Flutter version (fvm, Chapter 7.5) as local development — CI and a developer's machine are never running two different SDK versions.
- CI injects environment variables (Chapter 7.10) and Firebase service-account secrets (Chapter 7.7) from its own secret store, never from a committed file.
- A generated-code drift check (re-running build_runner and diffing against committed *.g.dart files, Chapter 7.12) fails the build if a contributor forgot to regenerate and commit.
- iOS build/sign/test runs via the cloud Mac service (Chapter 7.1, Section 4) as a distinct CI job, using the AWS/Firebase credentials handed off per Chapters 7.7/7.8 — vendor and exact configuration are an implementation detail owned by whoever sets up the pipeline, not re-litigated here.

3. Sign-Off Statement

Volume 7's goal — "create a reproducible development setup" — is considered met once a second person, following only Chapters 7.1–7.12 with no undocumented tribal knowledge, reaches every ✓ in Section 1 above on a clean Windows machine. This checklist is the acceptance test for the volume itself, not just for the app.